

Project Overview

- This project investigates social media interactions among members of the U.S. Congress can be used to identify influential political figures. Using a directed, weighted graph constructed from a Twitter interaction dataset, the project analyzes each member's degree centrality (their number of outgoing connections) to answer this question: *"Do the most socially active or mentioned members of Congress on Twitter correspond to those with real-world political influence?"*
- I used degree and centrality metric to visualize, quantify, and rank influence in a network context, exploring how digital presence may reflect or diverge from actual power.
- Dataset (Stanford Large Network Dataset Collection):
<https://snap.stanford.edu/data/congress-twitter.html>

Data Processing

- The dataset (`edgelist.txt`) was loaded using a custom Rust function `Graph::from_file`, which reads the file line by line using `BufReader`. Each line represents a directed edge in the form of: `from_node to_node weight`
- The graph is initialized with the total number of nodes, inferred by scanning the file for the highest node index using the `infer_graph_size` function.

Cleaning or Transitions applied

- Filtering malformed lines - I skipped the lines with fewer than 3 fields (missing `from`, `to`, or `weight`) during loading
- Parsing validation - Each line's fields were parsed into `usize` and `f64` types with error checking. Invalid lines (like non-numeric values) would be ignored.
- Edge weight conversion - The weights were converted from strings to `f64` for use in weighted graph metrics
- Adjacency matrix: After it was loaded, an adjacency matrix was generated for easier inspection and debugging.
- DataFrame creation: All the edges were collected into a Polars DataFrame with three columns: `from`, `to`, and `weight`, which stimulates a pandas-like structure
- Histogram binning: The node degrees were grouped into bins of size 5 for plotting histograms of connectivity

Code Structure

1. Modules

`main.rs`

Purpose: Leads the overall analysis pipeline

Responsibility: Loads the graph, builds the DataFrame, computes degree and centrality, prints summaries, saves outputs, and triggers visualizations

`graph.rs`

Purpose: Contains the graph representation and utility functions

Responsibilities: Builds the graph from file, computes adjacency matrix, calculated metrics, and generates visualizations and plots.

2. Key Functions & Types (Structs, Enums, Traits, etc)

Struct: `Graph`

Purpose: Represents a directed, weighted graph

Fields:

- `n`: `usize`: number of nodes
- `outgoing`: `Vec<Vec<(usize, f64)>>`: adjacency list for outgoing edges

Function: `Graph::from_file`

Input:

- `filename`: `&str` – path to file
- `n`: `usize` – number of nodes

Output:

- Graph instance

Logic:

- Parses each line into `from`, `to`, and `weight` values
- Skips malformed lines
- Builds the `outgoing` edge list

Function: `infer_graph_size`

Purpose: Finds the maximum node index to infer total graph size

Input: `filename`: `&str`

Output: `usize` (number of nodes)

Logic: Scans the file and finds the largest node ID

Function: `to_adjacency_matrix`

Purpose: Converts the graph into a 2D matrix for inspection

Output: `Vec<Vec<f64>>matrix`

Logic: Fills a square matrix with weights from outgoing edges

Function: `visualize_graph`

Purpose: Renders the graph into an image file (graph_visualization.png)

Inputs:

- `graph: &Graph`
- `centrality_scores: &Vec<(usize, f64)>`
- `top_n: usize`

Logic:

- Randomly assigns positions to nodes
- Draws edges and highlights top n central nodes in red

Function: `plot_histogram`

Purpose: Plot a binned histogram of out-degrees

Inputs: `HashMap<usize, usize>(bin → count)`

Output: Saves degree_histogram.png

Function: `plot_centrality_histogram`

Purpose: Writes node centralities to a file (centrality.csv)

Input: `Vec<(usize, f64)>`

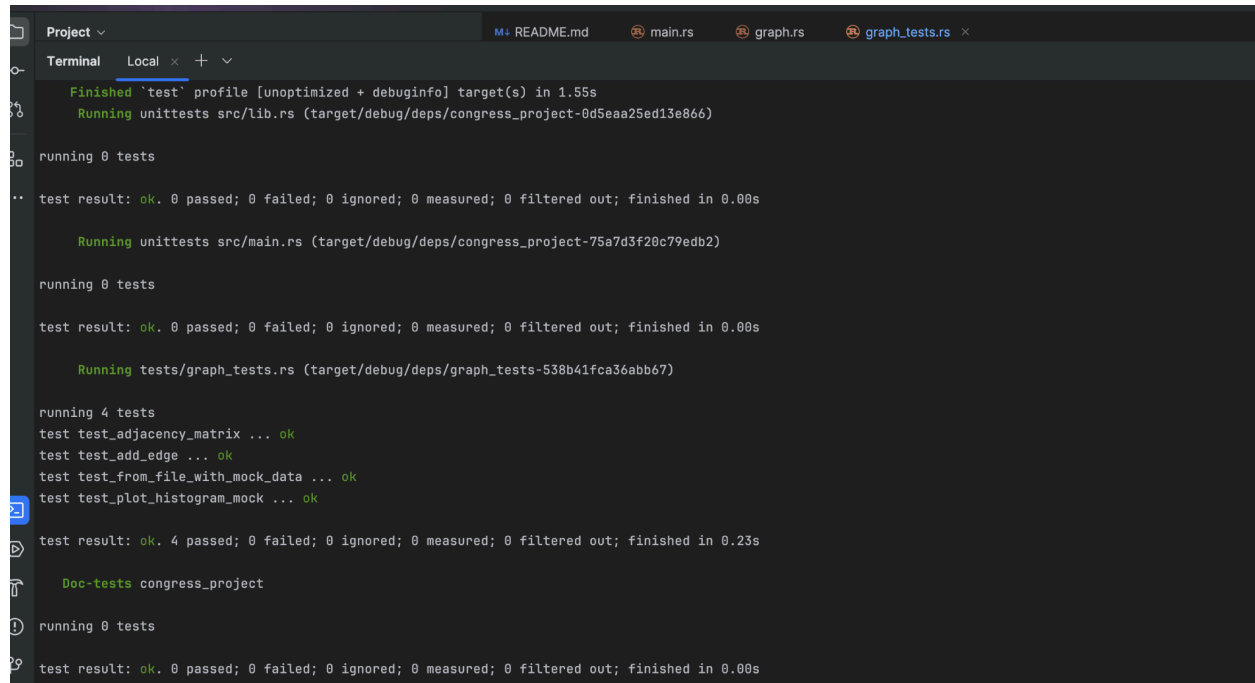
3. Main Workflow

At glance, the flow looks like this:

1. Load File → Build Graph
Infer_graph_size → Graph::from_file
2. Data Representation
 - a. Build Polars DataFrame from edges list
 - b. Generate adjacency matrix
3. Metrics + Ranking
 - a. Compute node degrees
 - b. Sort by out-degree
 - c. Normalize to degree centrality
4. Output and Visualizations
 - a. Print top 10 nodes by degree and centrality
 - b. Plot histograms
 - c. Visualize graph
 - d. Save centrality CSV

Tests

- cargo test output



```
Project v
Terminal Local x + v
Finished `test` profile [unoptimized + debuginfo] target(s) in 1.55s
Running unittests src/lib.rs (target/debug/deps/congress_project-0d5eaa25ed13e866)
running 0 tests
test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
Running unittests src/main.rs (target/debug/deps/congress_project-75a7d3f20c79edb2)
running 0 tests
test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
Running tests/graph_tests.rs (target/debug/deps/graph_tests-538b41fca36abb67)
running 4 tests
test test_adjacency_matrix ... ok
test test_add_edge ... ok
test test_from_file_with_mock_data ... ok
test test_plot_histogram_mock ... ok
test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.23s
Doc-tests congress_project
running 0 tests
test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

test_add_edge

What it checks: This test creates a small graph and calls `.add_edge(0, 1, 1.5)`. Then it asserts that node 0's outgoing list contains a single edge to node 1 with weight 1.5.

Why it matters: It confirms that edges are being stored correctly in the graph's adjacency list. This is foundational because if adding edges is broken, nothing else in the graph logic would work properly.

test_adjacency_matrix

What it checks: It adds a few edges and checks whether the graph's `to_adjacency_matrix()` function produces the correct 2D matrix representation.

Why it matters: The adjacency matrix is used for visual inspection and sanity checks in your `main.rs`. This test ensures that weights are correctly reflected in the matrix output.

test_from_file_with_mock_data

What it checks: It loads a small, mock edgelist from a file, builds the graph using `Graph::from_file`, and asserts that the correct number of edges were parsed and stored.

Why it matters: It validates that your file-reading logic is parsing nodes and weights correctly—critical since your entire project depends on real data from `edgelist.txt`.

test_plot_histogram_mock

What it checks: It creates a fake degree histogram and checks whether the `plot_histogram` function runs without error.

Why it matters: It ensures that your histogram-plotting logic works as expected, even when provided with dummy input. This avoids runtime failures during actual visual output.

Results

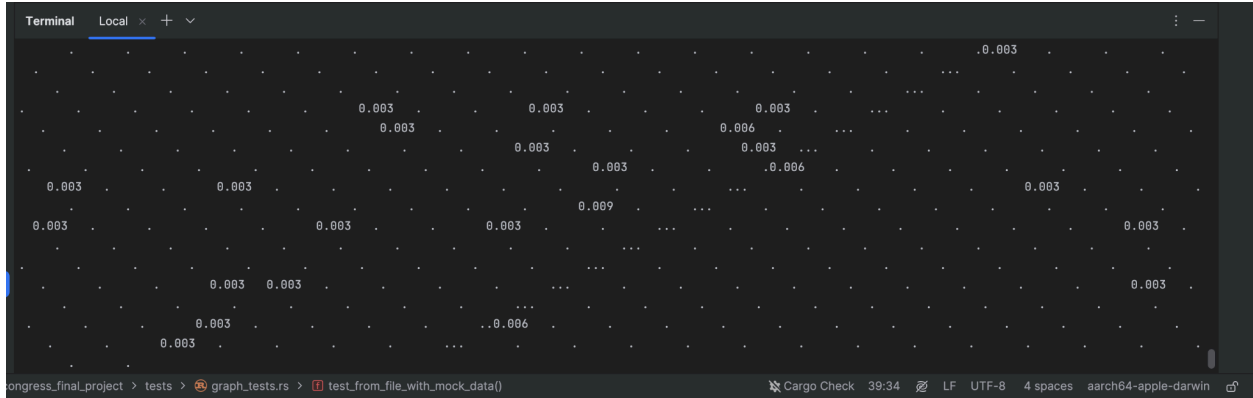
- All program outputs (screenshots or pasted)

```
Top 10 Most Connected Nodes:
Node 367: 210 outgoing connections
Node 322: 157 outgoing connections
Node 393: 111 outgoing connections
Node 71: 97 outgoing connections
Node 399: 89 outgoing connections
Node 436: 85 outgoing connections
Node 179: 84 outgoing connections
Node 254: 79 outgoing connections
Node 105: 75 outgoing connections
Node 87: 71 outgoing connections
```

```
Node with most outgoing connections: Node 367 with 210 edges.

Top 10 Most Influential Congress Members by Degree Centrality:
Node 367: Centrality 0.4430
Node 322: Centrality 0.3312
Node 393: Centrality 0.2342
Node 71: Centrality 0.2046
Node 399: Centrality 0.1878
Node 436: Centrality 0.1793
Node 179: Centrality 0.1772
Node 254: Centrality 0.1667
Node 105: Centrality 0.1582
Node 87: Centrality 0.1498

(base) annzhou@crc-dotlx-nat-10-239-186-10 congress_final_project %
congress_final_project > tests > graph_tests.rs > test_from_file_with_mock_data()
```



Interpretation in project context

- The analysis identifies Node 367 as the most connected member in the network, with 210 outgoing connections and a centrality score of 0.4430. This suggests that Node 367 frequently interacts with or mentions a large number of other congressional members on Twitter. Similarly, the other top-ranking nodes by degree centrality also have high out-degree counts, indicating a potentially prominent or active role in online political discourse.
- The nodes are not labeled with names, but the results demonstrate that some members show significantly higher connectivity than others, which suggests possible influence. This supports the broader goal of using social media interaction graphs to explore patterns of prominence or engagement among Congress members.
- Although this analysis doesn't establish real-world political power directly, it provides a framework to observe how certain figures stand out in digital spaces.

Usage Instructions

- How to build and run your code.
 - Clone or download the repository
 - `git clone git@github.com:annejiayinzhou/congress_final_project.git`
 - `cd congress_final_project`
 - Ensure the edgelist file exists: your graph data should be located at
 - `data/edgelist.txt`
 - Build the project
 - `cargo build`
- Description of any command-line arguments or user interaction in the terminal.
 - This program does not take any command-line arguments.
 - All output is printed directly to the terminal:
 - Graph size and structure
 - Top 10 most connected nodes
 - Top 10 centrality scores

- It also saves the following files:
 - `centrality.csv` — tabulated centrality scores
 - `degree_histogram.png` — binned out-degree plot
 - `centrality_histogram.png` — centrality distribution
 - `graph_visualization.png` — graph layout image
- Include expected runtime especially if your project takes a long time to run.
 - For the edgelist, the runtime is under 2 seconds
 - As for the larger graphs, the runtime is around 10-15 seconds because of them building the adjacency matrix, plotting with plotters, and randomizing graph layout

AI Statement

I didn't know how to graph a histogram in Rust, so I asked ChatGPT, and here is what it gave me:

To graph a histogram in Rust, the most common approach is to use the `plotters` crate. Below is a minimal working example that creates and saves a histogram as a PNG image.

Step-by-step: Drawing a Histogram in Rust

1. Add dependencies in `Cargo.toml`

```
toml

[dependencies]
plotters = "0.3"
```

2. Rust code to generate a histogram

This example bins a vector of `f64` values into integer buckets and plots them.

```
rust

use plotters::prelude::*;
use std::collections::HashMap;

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let data = vec![0.2, 0.5, 0.7, 0.5, 0.9, 0.8, 0.7, 0.2, 0.3, 0.4];

    // Bin the data
```

Ask anything

+ Search Deep research Create image ...

ChatGPT can make mistakes. Check important info.

```
use plotters::prelude::*;
use std::collections::HashMap;

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let data = vec![0.2, 0.5, 0.7, 0.5, 0.9, 0.8, 0.7, 0.2, 0.3, 0.4];

    // Bin the data
    let mut bins = HashMap::new();
    for &value in &data {
        let bin = (value * 10.0).floor() as usize; // bin width = 0.1
        *bins.entry(bin).or_insert(0) += 1;
    }

    let mut binned_data: Vec<(usize, usize)> = bins.into_iter().collect();
    binned_data.sort_by_key(|&(k, _)| k);

    // Create drawing area
    let root = BitMapBackend::new("histogram.png", (640, 480)).into_drawing_area();
    root.fill(&WHITE)?;

    let max_y = binned_data.iter().map(|&(_, y)| y).max().unwrap_or(1);

    let mut chart = ChartBuilder::on(&root)
        .caption("Histogram Example")
        .font(Font::new("sans-serif"))
```

Ask anything

+ Search Deep research Create image ...

ChatGPT can make mistakes. Check important info.

```
let mut bins: std::collections::HashMap<usize, usize> =
std::collections::HashMap::new();
```

```
for &(_ , centrality) in centrality_scores {
```

```
    let bin = (centrality * 100.0).floor() as usize;
```

```
    *bins.entry(bin).or_insert(0) += 1;
```



```
}
```

For this code, I group the centrality values into bins by multiplying by 100 and rounding down. This allows me to count how many nodes fall into each 1% range of centrality.

```
let mut data: Vec<(usize, usize)> = bins.into_iter().collect();  
  
data.sort_by_key(|&(bin, _)| bin);
```

This one turns the hash map into a sorted vector so that the bars in the histogram appear in order from low to high centrality.

```
let max_y = data.iter().map(|&(_, y)| y).max().unwrap_or(1);
```

Here, I find the maximum count across all bins to set the height of the histogram axes. If empty, I default to 1 to avoid errors when drawing the chart.

```
let root = BitMapBackend::new("centrality_histogram.png", (800,  
600)).into_drawing_area();  
root.fill(&WHITE)?;
```

Here, I create a 800x600 image where the histogram will be drawn, and fill the background white.

```
let mut chart = ChartBuilder::on(&root)  
    .caption("Degree Centrality Distribution", ("sans-serif", 30))  
    .margin(20)  
    .x_label_area_size(40)  
    .y_label_area_size(50)  
    .build_cartesian_2d(0u32..100u32, 0usize..(max_y + 5))?;
```

Here, I set up the plotting area with labeled axes and margins, and define the x-axis to range from 0 to 100% and the y-axis based on the largest bin count.

```
chart  
    .configure_mesh()  
    .x_desc("Degree Centrality (%)")  
    .y_desc("Number of Nodes")  
    .draw()?;
```

I label the axes to show what each direction means. X is the centrality and y is how many nodes fall into each bin

```
chart.draw_series(data.iter().map(|&(bin, count)| {  
    let bar_width = 2;
```

```
let x0 = bin as u32;  
let x1 = x0 + bar_width;  
  
Rectangle::new(  
    [(x0, 0), (x1, count)],  
    BLUE.filled(),  
)  
)))?;
```

I draw each bar in the histogram from (bin, 0) up to (bin+width, count). It means that each bar is filled with blue and represents how many nodes fall in that bin.